

DATASHARE

Documentation technique

Application de partage de fichiers

Angular • Spring Boot • PostgreSQL • Spring Security / JWT

Périmètre

MVP de partage de fichiers avec comptes utilisateurs, upload authentifié, upload anonyme (US07), liens de téléchargement, expiration, historique, suppression et protection facultative des fichiers par mot de passe (US09).

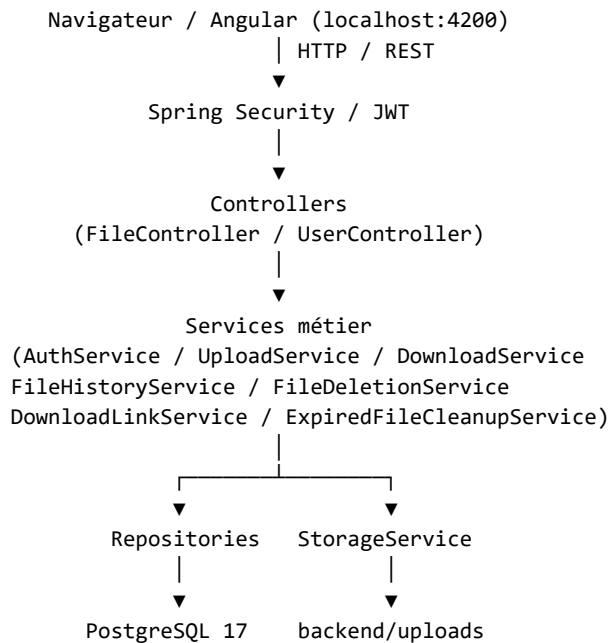
Version du document : 13 août 2026

Sommaire

1. Architecture de l'application
2. Choix technologiques justifiés
3. Modèle de données
4. Documentation d'API
5. Sécurité et gestion des accès
6. Qualité, tests et maintenance
7. Processus d'installation et d'exécution
8. Utilisation de l'IA dans le développement

1. Architecture de l'application

DataShare est une application web full-stack organisée en couches. Le frontend Angular dialogue avec une API REST Spring Boot. Le backend sépare les contrôleurs HTTP, les services métier, l'accès aux données JPA et le stockage physique des fichiers.



Responsabilités principales

- Angular : interface utilisateur, formulaires, appels HTTP, gestion du JWT côté client et affichage des retours de l'API.
- Controllors : exposition des endpoints REST et transformation des requêtes HTTP vers les services métier.
- Services : règles métier d'authentification, d'upload, de téléchargement, d'historique, de suppression, de génération/validation des liens et de nettoyage des fichiers expirés.
- Repositories JPA : persistance des entités User, StoredFile et DownloadLink dans PostgreSQL.
- StorageService / LocalStorageService : abstraction et implémentation du stockage physique des fichiers sur le système de fichiers.
- Spring Security : filtrage des requêtes, validation du JWT et contrôle des routes publiques/protégées.

Parcours d'un upload

Authentifié :

Angular → POST /api/files + JWT → FileController → UploadService
→ UserRepository → StorageService → StoredFileRepository
→ DownloadLinkService → DownloadLinkRepository → token / URL

Anonyme (US07) :

Angular → POST /api/files/anonymous sans JWT → FileController
→ UploadService.uploadAnonymous() → owner = null
→ StorageService → StoredFileRepository → DownloadLinkService
→ DownloadLinkRepository → token / URL

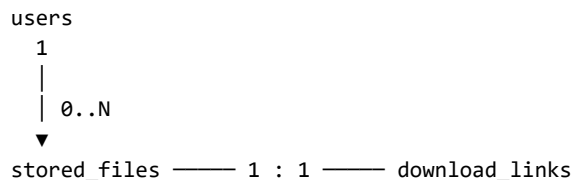
Les deux parcours partagent le même traitement métier. La différence principale est l'association à un propriétaire : un fichier authentifié est rattaché à un User ; un fichier anonyme est enregistré avec une relation utilisateur facultative.

2. Choix technologiques justifiés

Technologie	Rôle	Justification
Angular / TypeScript	Frontend	Expérience sur l'outil. Framework structuré, typage TypeScript, séparation composants/services et intégration naturelle avec une API REST.
Java 21 / Spring Boot	Backend	Expérience sur l'outil. Écosystème robuste pour API REST, injection de dépendances, validation, sécurité et tests.
Spring Security	Sécurité	Centralise l'authentification, le filtrage JWT et les règles d'accès aux endpoints.
JWT	Authentification	Authentification stateless adaptée à une SPA : chaque requête protégée transporte son jeton.
BCrypt	Mots de passe	Hashage non réversible des mots de passe utilisateurs et des mots de passe facultatifs des fichiers.
Spring Data JPA	Persistance	Abstraction des opérations CRUD et des requêtes sur les entités.
PostgreSQL 17	BDD	Base relationnelle adaptée aux relations User / StoredFile / DownloadLink et exécutée de façon reproductible avec Docker.
Docker Compose	Infrastructure	Démarrage simple et reproductible de PostgreSQL avec volume persistant.
JUnit 5 / Mockito / MockMvc	Tests backend	Expérience sur l'outil. Tests unitaires et contrôleurs avec isolation des dépendances.
Cypress	E2E	Expérience sur l'outil. Validation des parcours critiques depuis l'interface jusqu'au backend.
JaCoCo	Couverture	Expérience sur l'outil. Mesure de la couverture du code backend.
k6	Performance	Test de charge léger reproductible sur un parcours authentifié.

3. Modèle de données

La base PostgreSQL contient trois tables métier principales. Le fichier binaire n'est pas stocké en base : StoredFile contient les métadonnées et la référence permettant de retrouver le fichier physique via StorageService.



Pour US07 (Optionnel) :
stored_files.user_id peut être NULL

Pour US09 (Optionnel) :
stored_files.password_hash peut être NULL

Table	Rôle	Données principales
users	Compte utilisateur	Identité, email, mot de passe hashé.
stored_files	Métadonnées du fichier	Nom original, nom de stockage, MIME, taille, dates, propriétaire facultatif, password_hash facultatif.
download_links	Accès par token	Token unique, expiration et association au StoredFile.

Séparation métadonnées / binaire

PostgreSQL conserve les informations métier. Le contenu physique (.pdf, .txt, .docx, etc.) est stocké séparément dans backend/uploads/ via LocalStorageService.

4. Documentation d'API

L'API REST est exposée par le backend sur <http://localhost:8080>. Les routes protégées utilisent le header Authorization: Bearer <JWT>.

Méthode	Endpoint	Accès	Fonction
POST	/api/user	Public	Créer un compte utilisateur.
POST	/api/login	Public	Authentifier l'utilisateur et retourner un JWT.
POST	/api/files	JWT	Uploader un fichier pour l'utilisateur connecté ; multipart/form-data.
POST	/api/files/anonymous	Public	Uploader un fichier sans compte (US07) ; multipart/form-data.
GET	/api/files	JWT	Récupérer l'historique des fichiers de l'utilisateur connecté.
GET	/api/files/{token}	Public	Récupérer les métadonnées associées à un lien de téléchargement.
POST	/api/files/{token}/file	Public par token	Télécharger le fichier ; accepte un mot de passe dans le body si le fichier est protégé.
DELETE	/api/files/{id}	JWT	Supprimer un fichier appartenant à l'utilisateur connecté.
GET	/api/health	Public	Vérifier la disponibilité de l'API (pour ELK)

Pourquoi POST pour le téléchargement protégé ?

US09 nécessite de transmettre éventuellement un mot de passe. Le téléchargement utilise donc POST afin d'envoyer ce secret dans le corps de la requête plutôt que dans l'URL. Une évolution possible serait de conserver en parallèle un GET direct pour les fichiers non protégés.

```
POST /api/files/{token}/file
Content-Type: application/json
```

```
{
  "password": "mot-de-passe-eventuel"
}
```

Mot de passe absent/incorrect sur un fichier protégé → HTTP 403

5. Sécurité et gestion des accès

Authentification JWT stateless

```
LOGIN
email + mot de passe
↓
AuthService
↓
BCrypt vérifie le mot de passe utilisateur
↓
JwtService génère le JWT
↓
Angular conserve le jeton

REQUÊTE PROTÉGÉE
Angular
↓ Authorization: Bearer <JWT>
Spring Security
↓
JwtAuthenticationFilter
↓
JwtService valide le token
↓
SecurityContext / Authentication
↓
Controller → Service
```

Le backend ne conserve pas de session utilisateur classique : chaque requête protégée apporte le JWT nécessaire à son authentification.

Protection des fichiers - US09

- Le mot de passe du fichier est facultatif et doit contenir au moins 6 caractères lorsqu'il est renseigné.
- Il est hashé avec BCrypt avant persistance ; le mot de passe original n'est jamais stocké.
- DownloadService compare le mot de passe saisi au hash avec PasswordEncoder.matches().
- Un mot de passe absent ou incorrect pour un fichier protégé déclenche InvalidFilePasswordException et une réponse HTTP 403.
- Le frontend n'effectue pas la validation de sécurité : il affiche le message correspondant à la réponse du backend.

Routes publiques et protégées

- Publiques : création de compte, login, upload anonyme, téléchargement par token et health check.
- Protégées par JWT : upload authentifié, historique et suppression.
- L'upload anonyme ne crée pas un faux utilisateur : il utilise owner = null.

Audits de dépendances

Le frontend a été contrôlé avec npm audit. Le scan initial signalait 12 vulnérabilités ; après npm audit fix, il restait 1 vulnérabilité faible, 3 modérées et aucune haute. Le backend a été analysé avec OWASP Dependency-Check 12.2.2 et un rapport HTML a été généré.

6. Qualité, tests et maintenance

Tests backend

La suite backend utilise JUnit 5, Mockito et MockMvc. Les tests couvrent notamment les services métier, les contrôleurs, le JWT, le filtre d'authentification et la configuration de sécurité.

Indicateur	Résultat
Tests backend	50
Failures	0
Errors	0
Couverture JaCoCo	72 %
Objectif indicatif	≥ 70 % atteint

Commandes :

```
cd backend
mvn clean test
```

Chemin du rapport :

```
backend/target/site/jacoco/index.html
```

Tests End-to-End Cypress

- authenticated-upload.cy.ts : connexion puis upload authentifié.
- anonymous-upload.cy.ts : upload anonyme puis téléchargement.
- password-protected-file.cy.ts : refus d'un mauvais mot de passe puis validation du bon parcours.

Commandes :

```
cd frontend
npx cypress run
```

Performance k6

Mesure	Résultat
Utilisateurs virtuels	10
Durée	30 s
Itérations	280
Requêtes HTTP	560
Échecs HTTP	0 %
Checks	840 / 840 (100 %)
Temps moyen	48,74 ms

p95	117,52 ms
Maximum	146,67 ms
Budget	< 500 ms respecté

Chaque itération réalise un POST /api/login, récupère le JWT puis appelle GET /api/files. Ce test constitue une référence de non-régression sous charge légère, et non un test de capacité maximale.

Commandes :

```
k6 run perf/files-history.js
# ou pour conserver le résultat :
k6 run perf/files-history.js | tee perf/k6-results.txt
```

Maintenance

```
Modifier le code
↓
Compiler
↓
Tests backend
↓
Cypress si parcours impacté
↓
Audit sécurité / k6 si nécessaire
↓
git status / git diff
↓
Commit Git
```

- Backend : dépendances gérées dans pom.xml ; rejouer mvn clean test après évolution.
- Frontend : dépendances dans package.json/package-lock.json ; privilégier npm ci pour une installation reproductible.
- Éviter npm audit fix --force sans analyser les changements majeurs.
- Ne pas versionner node_modules/, backend/target/, fichiers temporaires, téléchargements Cypress, secrets ou fichiers :Zone.Identifier.

7. Processus d'installation et d'exécution

Prérequis

- Linux ou WSL2
- Java 21
- Maven
- Docker / Docker Compose
- Node.js et npm

Environnement de développement de référence : Java 21.0.11, Maven 3.9.12, Node.js 22.22.1, npm 9.2.0 et PostgreSQL 17.

1. Démarrer PostgreSQL

Commandes :

```
cd backend
docker compose up -d
docker ps
```

Conteneur attendu : datashare-postgres
Port : localhost:5432

2. Démarrer le backend

Commandes :

```
cd backend
mvn spring-boot:run
```

API : http://localhost:8080

3. Installer et démarrer Angular

Commandes :

```
cd frontend
npm install
npm start
```

Frontend : http://localhost:4200

4. Vérifications recommandées

Commandes :

```
# Backend
cd backend
mvn clean test
```

```
# Frontend E2E
cd frontend
npx cypress run
```

```
# Audit frontend
npm audit
```

```
# Performance (depuis la racine)
k6 run perf/files-history.js
```

8. Utilisation de l'IA dans le développement

L'IA a été utilisée comme outil d'accompagnement au développement et à la documentation. Elle n'a pas remplacé l'exécution, la compilation ni la validation technique du projet.

- Explication de notions Java, Spring Boot, Angular, REST, JWT et tests.
- Aide à l'analyse d'erreurs de compilation, d'exécution et de tests.
- Relecture de la séparation des responsabilités entre controllers, services, repositories et stockage.
- Accompagnement pour US07 (upload anonyme) et US09 (protection facultative par mot de passe).
- Aide à la conception et à l'amélioration des tests JUnit/Mockito/MockMvc et Cypress.
- Interprétation des rapports JaCoCo, k6, npm audit et OWASP Dependency-Check.
- Structuration des documents TESTING.md, SECURITY.md, PERF.md, MAINTENANCE.md et de la documentation technique.

Principe de validation

Les propositions issues de l'IA ont suivi le cycle : compréhension → implémentation → compilation → tests → validation dans l'environnement réel. Le code final et les résultats présentés correspondent aux éléments effectivement exécutés dans le projet.

Conclusion

DataShare repose sur une architecture en couches claire et testable, avec une API REST Spring Boot sécurisée par JWT, un frontend Angular, PostgreSQL pour les métadonnées et un stockage local abstrait pour les fichiers. Les fonctionnalités optionnelles US07 et US09 ont été intégrées sans dupliquer le cœur de l'upload. Le projet est accompagné de tests automatisés, d'une mesure de couverture, d'un test de performance, d'audits de dépendances et de procédures de maintenance.